

Сборка паков ассетов: инструкция для пользователя

Эта инструкция помогает получить паки ассетов для конкретной версии игры, проверить их и подготовить комплект для `/app0`. Основной путь — оконная программа `ampr_pack_gui.py`; команды PowerShell приведены в конце как запасной и диагностический вариант.

Главное ограничение трассировки

Трассы **не содержат все данные игры**. Они показывают только APR-чтения, реально выполненные во время записанных сценариев. В них не будет файлов из непройденных уровней, других режимов и языков, части DLC, редких веток, а также обращений через неперехваченные пути вроде `mmap`.

Автоматически полученный TOML — только начальная заготовка. Перед сборкой его нужно адаптировать под реальную игру: добавить оставшиеся нужные файлы и папки либо сознательно оставить их обычными loose-файлами. Даже длинная трасса не доказывает, что не встреченные в ней файлы никогда не понадобятся.

1. Что понадобится

- Windows и Python 3.11 или новее с компонентом Tcl/Tk. В обычной установке Python с python.org Tkinter уже присутствует.
- Полная неизменённая копия игровой папки `/app0` на ПК.
- `ampr_emu.index`, построенный именно для этой копии `/app0`.
- Одна или несколько трасс. В каталоге каждого прогона должны лежать рядом `ampr_commands.bin` и соответствующий ему `ampr_emu.index`.
- Свободное место одновременно для оригиналов, предыдущего комплекта паков и нового временного комплекта. При пересборке не рассчитывайте только на размер конечных `.pak`: до публикации старые и новые тома могут существовать вместе.

Сначала установите зависимость упаковщика. Откройте PowerShell в корне репозитория и выполните:

```
python -m pip install -r tools/requirements-pack.txt
```

Не смешивайте журнал одной версии игры с индексом другой версии. `fileId` зависит от записей AMPRIDX3, поэтому рядом с каждой трассой храните индекс, который использовался при её записи.

Если для полной локальной копии индекс ещё не создан, постройте его один раз и после этого не меняйте до завершения всех трасс и упаковки:

```
python tools/build_ampr_index.py "profiling/title/app0"
```

Рекомендуемая структура рабочего каталога:

```
profiling/title/
├─ app0/                                полная исходная игра
│   └─ ampr_emu.index
├─ traces/
│   ├── startup/
│   │   ├── ampr_commands.bin
│   │   └─ ampr_emu.index
│   ├── level-and-combat/
│   │   ├── ampr_commands.bin
│   │   └─ ampr_emu.index
│   └─ travel/
│       ├── ampr_commands.bin
│       └─ ampr_emu.index
├─ profiles/
└─ packed/
```

2. Как получить полезные трассы

Для сбора трасс используется готовая **отладочная версия эмулятора** с включённой записью APR-команд.

Запишите несколько отдельных сценариев:

1. Холодный запуск до главного меню и загрузка сохранения.
2. Загрузка нескольких разных уровней или больших областей.
3. Fast travel, кат-сцены, бой и интенсивная потоковая подгрузка.
4. Повторное посещение области.
5. Дополнительные режимы, языки и установленное DLC, которые должен поддерживать итоговый комплект.

После штатного завершения каждого прогона скопируйте в отдельный каталог

`ampr_commands.bin` , `ampr_emu.index` и, для диагностики, `ampr_emu.log` .

Оборванный после аварии журнал может быть неполным.

Количество прогонов улучшает рекомендации по геометрии и кэшам, но не отменяет ручную проверку состава игры.

3. Запустить GUI

На Windows дважды щёлкните:

```
tools\ampr_pack_gui.cmd
```

Или запустите из PowerShell:

```
python tools/ampr_pack_gui.py
```

В верхней части окна заполните пять путей:

- **Папка с трассами** — общий каталог, внутри которого GUI найдёт все пары `ampr_commands.bin` + `ampr_emu.index` .
- **Полная копия /app0** — исходные файлы игры, не каталог с уже удалёнными оригиналами.
- **Индекс `ampr_emu.index`** — индекс этой полной копии игры.
- **Профиль TOML** — куда сохранить сгенерированные правила, например `profiling/title/profiles/title.toml` .
- **Папка готовых паков** — отдельный каталог результата, например `profiling/title/packed` .

Нажмите **«1. Создать профиль из трасс»**. GUI объединит все найденные прогоны и рядом с TOML сохранит:

- `title.report.md` — читаемый отчёт;
- `title.metrics.json` — подробные входные метрики;
- при необходимости списки `*.include.txt` .

JSON описывает входные обращения и рекомендации. Это не измерение скорости готового packed runtime.

Как читать отчёт профилировщика

В `title.report.md` сначала проверьте `warnings` и сведения о каждом входном прогоне. Ненулевые `Sequence/parser warnings` или `decode issues` должны быть объяснены; повреждённую или оборванную трассу лучше переснять, чем строить на ней агрессивные правила.

Поле отчёта	Как его использовать
Confidence	При низкой уверенности не делайте правило агрессивнее без новых трасс; безопасная отправная точка — <code>64KiB/mixed</code>
Amplification	Оценивает, во сколько раз выбранная геометрия затрагивает больше данных, чем запросила игра; сравнивайте варианты одного файла
Seq.	Высокая доля последовательного доступа обосновывает <code>streaming</code> и более крупные блоки
Repeat 64K	Показывает повторное обращение к областям и помогает оценить пользу <code>decoded-кэша</code>
Projected pack index	Прогноз резидентного манифеста только для состава на момент генерации
Recommended internal pool	Расчёт для рекомендованных кэшей, <code>workers</code> и прогнозного манифеста, а не автоматически изменённый размер <code>PRX</code>

После ручного добавления каталогов старые `Projected pack index` и `Recommended internal pool` больше не являются итоговыми. Ориентируйтесь на фактический `ampr_assets.index` после сборки и заново проверяйте бюджет памяти.

4. Обязательно дополнить профиль под реальную игру

После генерации нажмите **«Загрузить из профиля»**. В секции **«Все пути, выбранные для упаковки»** появятся вместе все явные `include` и `include_from` из трассовых правил `compress / store`, а также ранее сохранённые ручные добавления. Для приложенного профиля это позволяет увидеть весь сгенерированный список, а не пустое поле ручных путей.

Сравните этот список с настоящим содержимым `/app0` и поддерживаемыми игровыми сценариями. Отредактируйте выбор:

- **«Добавить файлы...»** — выбирает один или несколько конкретных файлов;
- **«Добавить папку...»** — добавляет весь каталог как `путь/**` ;
- **«Добавить маску...»** — добавляет путь или glob вручную, например `dlc/**` или `localization/ru/**` .
- **«Удалить выбранное»** — удаляет путь из итогового выбора. Для пути из трассового правила исходное правило и его параметры сохраняются, а GUI записывает последнее `loose` -исключение;
- **«Загрузить из профиля»** — повторно строит полный редактируемый список из выбранного TOML и его безопасных относительных `include_from` .

Выбранные пути обязаны находиться внутри указанного `/app0` . GUI записывает их в `title.manual.include.txt` и добавляет в TOML управляемое правило. Оно не отменяет жёсткие исключения для `eboot.bin` , `PRX/SPRX`, индексов и системных каталогов.

Параметры новых ручных путей

Параметр GUI	Значение	Когда использовать
Режим: <code>compress</code>	Независимо сжимает блоки LZ4 HC уровня 12 по умолчанию; несжимаемый блок сохраняется как RAW	Начальный вариант для большинства игровых данных; сборка медленнее <code>fast</code> , но размер обычно меньше
Режим: <code>store</code>	Не тратит CPU на LZ4, но помещает данные в seekable-пак; CRC декодированных блоков сохраняются только в offline-sidecar	Уже сжатые видео, аудио и архивы после проверки
Доступ: <code>mixed</code>	Сочетает ограниченный произвольный доступ и объединение соседних чтений	Рекомендуемый начальный вариант
Доступ: <code>random</code>	Оптимизирует небольшие непоследовательные чтения	Таблицы, скрипты, конфигурация и мелкие часто используемые файлы
Доступ: <code>streaming</code>	Плотно размещает последовательные блоки	Большие видео/аудио или доказанно последовательные данные
Блок	Размер независимо читаемого и декодируемого блока: 16 KiB — 1 MiB	Начинайте с <code>64kiB</code> ; крупнее — для <code>streaming</code> , мельче — для мелких <code>random</code> -чтений

Поля **«Режим»**, **«Доступ»** и **«Блок»** применяются только к новым путям, которых не было в загруженных трассовых правилах. Параметры уже существующих правил не уплощаются и не меняются; для изменения их геометрии отредактируйте соответствующий `[[rule]]` в TOML вручную.

GUI добавляет к правилу новых путей `force_pack = true` : явно выбранные файлы не возвращаются автоматически в loose из-за слабого сжатия, а несжимаемые блоки остаются RAW. Жёсткие системные исключения всё равно имеют приоритет.

Когда выбирать `compress` , `store` или `loose`

Действие	Выбирайте, если	Основной риск
<code>compress</code>	Данные сжимаются или содержат смесь сжимаемых и RAW-блоков	LZ4 требует CPU; слабое сжатие может не окупить декодирование
<code>store</code>	Данные уже сжаты, но объединение в тома полезно для размещения и сокращения числа файлов/FD	Размер почти не уменьшится, а чтение всё равно зависит от packed runtime
<code>loose</code>	Возможны <code>mmap</code> или другой неперехваченный доступ, файл редко нужен либо большой файл почти не сжимается и упаковка не даёт измеримого выигрыша	Остаётся обычная файловая нагрузка и требуется сохранить оригинал на диске

`force_pack = true` отключает автоматический возврат явно выбранного большого несжимаемого файла в loose. Используйте его только когда packed-доступ нужен осознанно; RAW fallback сохраняет корректность данных, но не устраняет I/O и служебные расходы packed runtime.

Файлы с общим префиксом необязательно должны иметь одинаковую политику. Например, базовый soundbank можно оставить loose, а локализованные варианты упаковать:

```
[[rule]]
action = "compress"
include = ["d/soundbank.*"]
block_size = "64KiB"
layout = "mixed"
force_pack = true

[[rule]]
action = "loose"
include = ["d/soundbank"]
```

Если пробное сжатие локализованных банков не даёт выигрыша, замените для них `compress` на `store` либо оставьте их `loose`. Точные правила-исключения размещайте после пересекающихся широких правил.

Для уже сжатых потоковых видео/аудио можно начать со `store`, `streaming` и `256kiB`, но менять геометрию стоит только с последующим игровым A/B-тестом. Если разным наборам нужны разные параметры, сохраните основной список через GUI, а затем добавьте отдельные `[[rule]]` в TOML вручную. Правила применяются сверху вниз; побеждает последнее совпавшее правило.

Как правильно подобрать размер блока

Размер блока выбирайте по характеру чтений в трассах, а не только по расширению или имени файла. Смотрите на типичный размер запроса, долю случайных чтений, повторное использование данных и оценку `read amplification` в отчёте профилировщика.

Размер блока	Когда это разумная отправная точка
16KiB	Трассы подтверждают очень мелкие случайные чтения горячих таблиц, конфигурации или локализации
32KiB	Преобладают небольшие random-чтения, но накладные расходы 16kiB уже нежелательны
64KiB	Безопасный вариант для неизвестных и смешанных данных, а также архивов с произвольными seek; совпадает с гранулой учёта APR
128KiB – 256KiB	Чтения в основном крупные или последовательные, что подтверждено трассами

Размер блока	Когда это разумная отправная точка
512KiB – 1MiB	Доказанно последовательные потоки и уже сжатые RAW-медиа; для мелких random-чтений обычно слишком крупно

Мелкий блок уменьшает объём лишнего чтения и декодирования при случайном доступе, но не является бесплатной оптимизацией. Каждый chunk занимает 12 байт в резидентном манифесте `AMPRPAK4`. Приближённая оценка только таблицы chunks:

```
chunk_count    ~= ceil(суммарный_размер_файлов / block_size)
chunk_metadata ~= chunk_count * 12 байт
```

Например, для 100 GiB упакованных логических данных таблица chunks занимает примерно 75 MiB при 16KiB, 37,5 MiB при 32KiB, 18,75 MiB при 64KiB и 1,17 MiB при 1MiB. Кроме роста манифеста, слишком мелкие блоки создают больше единиц планирования и операций копирования/декодирования и обычно ухудшают степень сжатия. Offline-sidecar CRC также растёт на 4 байта на chunk, но runtime его не загружает. Задержка может вырасти, даже если read amplification уменьшилась.

Не применяйте 16KiB или 32KiB широкой маской ко всему большому каталогу без трассового подтверждения: несколько десятков гигабайт ранее неучтённых файлов могут неожиданно занять десятки мегабайт внутреннего пула только таблицей chunks. Не дублируйте одну широкую маску в правилах с разным размером блока: побеждает последнее совпадение, поэтому оно может незаметно отменить точные рекомендации профилировщика.

Жёсткие лимиты формата и runtime

Проверяйте не только размер манифеста, но и количество записей. Текущая packed-сборка имеет следующие пределы:

Объект	Предел
Файлы в <code>AMPRPAK4</code>	2 000 000
Chunks во всех packed-файлах	16 000 000
Тома <code>.pak</code> во всём манифесте	1024
Группы упаковки	256
Размер блока	от 16 KiB до 1 MiB, степень двойки

Объект	Предел
Runtime workers	от 1 до 16
Запрос decoded-кэша	не более 512 MiB
Запрос physical-кэша	не более 128 MiB

Лимит томов относится ко всему манифесту, а не к одной `[groups.*]`. Даже если парсер TOML принял большее `pack_count`, такой комплект runtime не загрузит. Например, 250 GiB с блоком `16kiB` создают около 16,38 млн chunks ещё без других файлов и уже превышают предел. Укрупните блоки или оставьте часть данных loose.

Группы, lanes и striping

Для небольшого набора начинайте с одного тома в группе. Для большой группы с несколькими одновременно читаемыми файлами обычно достаточно двух–четырёх lanes и `assignment = "balanced"`. Большее число томов не ускоряет один небольшой запрос и может увеличить переключения FD и seek-нагрузку.

Оставляйте `stripe_large_files = false`, пока консольный A/B-тест не показал пользу чередования частей одного большого файла. При включении подбирайте `stripe_group_blocks` так, чтобы одна stripe была около 512 KiB. Не включайте striping для строго последовательного одиночного потока или хранилища, сериализующего все чтения.

Если lanes создаются ради параллельного I/O, сохраняйте `deduplicate_scope = "lane"`: дедупликация на всю группу может свести одинаковые данные в один том и уменьшить параллелизм. Не задавайте слишком маленький `max_pack_size` — это создаёт лишние тома; выбирайте ограничение из требований носителя и процесса развёртывания, контролируя общий предел 1024.

Особенно проверьте:

- уровни, карты и биомы, которых не было в записанных прогонах;
- все поддерживаемые языки, озвучки и субтитры;
- DLC, бонусные режимы, новые сохранения и финальные игровые главы;
- данные для кат-сцен, меню, титров и восстановления после ошибки;

Когда проверка закончена, нажмите **«2. Сохранить изменения путей»** и поставьте флажок подтверждения под списком. Этот флажок не доказывает полноту автоматически; он защищает от случайной сборки сразу после генерации трассового профиля.

5. Собрать и проверить паки

Нажмите **«3. Собрать и проверить»**. GUI последовательно выполнит:

1. Сборку томов, `ampr_assets.index` и offline-файла `ampr_assets.index.crc`.
2. Проверку каждого блока и декодирования с CRC из `offline-sidecar`.
3. Побайтовое сравнение восстановленных packed-файлов с исходным `/app0`.
4. Вывод сводки манифеста.

Во время сборки журнал GUI показывает строки прогресса `ampr_pack.py`: текущий этап, общий процент, число обработанных файлов, объём обработанных исходных данных, текущий путь и прошедшее время. Прогресс обновляется также внутри большого одиночного файла, поэтому HC12-сжатие не выглядит зависшим.

Этап считается успешным только при завершении всех четырёх операций без ошибки. Кнопки **«Проверить готовые паки»** и **«Показать сводку»** позволяют повторить проверку отдельно. Полный вывод остаётся в нижней части окна.

Не удаляйте исходные файлы после одного успешного `verify`: он доказывает целостность созданных паков, но не полноту игровых сценариев и не перехват всех способов чтения.

Контрольный список готовности

До переноса комплекта и особенно до удаления оригиналов должны выполняться все условия:

- ☐ предупреждения и `decode issues` входных трасс отсутствуют либо каждое из них объяснено;
- ☐ ручной выбор покрывает нужные уровни, режимы, локали и DLC;
- ☐ `pack`, `verify --root`, `inspect` и `list --json` завершились без ошибок;
- ☐ `loose_paths` просмотрен, и каждый указанный файл сохранён в `/app0`;
- ☐ фактические количества файлов, `chunks` и томов ниже жёстких лимитов;
- ☐ фактический размер манифеста и runtime-настройки помещаются во внутренний пул с запасом не менее 8–16 MiB;
- ☐ индекс, `.runtime` и все тома имеют один build ID и перенесены runtime-комплектom;
- ☐ соответствующий файл `.crc` сохранён рядом с индексом в архиве/на ПК для `verify` и `unpack`;
- ☐ полный консольный тест сначала выполнен при сохранённых оригиналах;
- ☐ подготовлены резервная копия и проверенный способ отката.

Если хотя бы один пункт не выполнен, комплект ещё не готов к удалению исходных файлов. Успешный `offline verify` закрывает только проверку целостности.

6. Удалить оригиналы, попавшие в паки

После сохранения резервной копии GUI может удалить из выбранной `/app0` только файлы, которым итоговый манифест присвоил статус PACK. Нажмите **«Удалить из /app0 файлы со статусом PACK...»**.

Перед удалением GUI показывает число файлов и общий объём. После подтверждения он заново выполняет полную проверку паков и побайтовое сравнение с оригиналами. Если манифест, пак, размер или содержимое исходного файла не совпадает, удаление не начинается.

Параметры удаления:

Параметр	Назначение
Полная копия /app0	Каталог, из которого удаляются оригиналы. Пути берутся только из PACK-записей выбранного манифеста
Папка готовых паков	Содержит проверяемый <code>ampr_assets.index</code> и его тома; LOOSE-записи не удаляются
Подтверждение проверки покрытия	Обязательно: <code>offline verify</code> не доказывает, что трассы охватили всю игру или что файл не читается через <code>mmap</code>
Удалить также опустевшие каталоги	После файлов удаляет только действительно пустые каталоги; корень <code>/app0</code> никогда не удаляется

Операция дополнительно отказывается удалять `eboot.bin`, `PRX/SPRX/SELF/ELF`, индексы AMPR, системные каталоги, `symlink/reparse point` и любой путь, выходящий за пределы выбранной `/app0`, даже если ошибочный пользовательский TOML пометил его как PACK.

Удаление безвозвратное. После него `verify --root` уже не сможет сравнить паки с исходниками. Храните отдельную резервную копию. Восстановить packed-файлы также можно командой `unpack`, пока сохранены исправные индекс, его `.crc` и все тома.

Перед запуском игры убедитесь, что `ampr_assets.index`, `.runtime` и все тома уже добавлены в итоговый комплект `/app0`. Само удаление их туда не копирует.

7. Что перенести в игру

В `/app0` переносится согласованный комплект одной сборки:

- исходный `ampr_emu.index` ;
- `ampr_assets.index` ;
- `ampr_assets.index.runtime` ;
- все `.pak` , перечисленные манифестом;
- все файлы, оставшиеся `loose`.

`ampr_assets.index.crc` рантайму не нужен и в память не загружается. Храните его рядом с архивной/PC-копией индекса: Python-команды `verify` и `unpack` требуют совпадающие build ID и число chunks. Развернуть `.crc` в `/app0` можно для удобства обслуживания, но это не влияет на игру.

Не смешивайте индекс, `.runtime` и тома от разных сборок: runtime-настройки привязаны к build ID манифеста. Обновляйте комплект только при остановленной игре.

Собирайте новый комплект в отдельном staging-каталоге. Во время пересборки закладывайте место как минимум под исходную игру, существующий комплект и новый комплект с временными файлами. Встроенная публикация защищает отдельный каталог от обычной ошибки сборки, но копирование файлов на другой носитель не становится атомарным автоматически.

Для обновления остановите игру, полностью скопируйте новый согласованный комплект, проверьте наличие всех перечисленных томов и только затем переключайте его целиком. Не заменяйте `.runtime` , индекс или тома по одному в работающей игре. До завершения консольной проверки сохраняйте предыдущий рабочий комплект. Для воспроизводимости архивируйте TOML, все `include_from` , отчёт, исходный `ampr_emu.index` и сводку `inspect` ; контрольные суммы итоговых файлов позволяют отличить неполное копирование от ошибки runtime.

Для первых тестов сохраняйте все оригиналы и используйте отладочную версию эмулятора с поддержкой `packed`-файлов. Удаление оригинала допустимо только после проверки всех игровых путей и только для файлов со статусом `PACK`, которые не читаются перехватываемым способом. Никогда не удаляйте модули, исполняемые файлы, индексы и настройки.

8. Параметры runtime-конфигурации

Профиль TOML содержит секцию `[runtime]` . При сборке из неё создаётся файл `ampr_assets.index.runtime` , привязанный к build ID конкретного манифеста:

```
[runtime]
decoded_cache_bytes = "128MiB"
physical_cache_bytes = "32MiB"
workers = 4
latency_reserve_workers = 1
```

Параметр	Назначение и допустимые значения
decoded_cache_bytes	Кэш уже декодированных блоков. Кратен 16 KiB; 0 отключает кэш. Увеличение полезно при повторных чтениях, но расходует общий внутренний пул
physical_cache_bytes	Кэш считанных физических страниц паков. Кратен 16 KiB; 0 отключает кэш. Помогает при повторном использовании одной страницы разными блоками
workers	Число рабочих потоков packed runtime: 1..16 . Это не то же самое, что [pack].workers , управляющий компрессией на ПК
latency_reserve_workers	Число workers, резервируемых для чувствительных к задержке чтений: от 0 до workers - 1

GUI использует эти значения при сборке паков. Чтобы изменить только runtime после сборки, отредактируйте [runtime] в TOML и обновите .runtime без перекомпрессии:

```
python tools/ampr_pack.py runtime-config --index "profiling/title/packed/ampr_assets.index" --config
```

Перенесите обновлённый ampr_assets.index.runtime и перезапустите игру. Повреждённый .runtime или файл от другого build ID блокирует загрузку манифеста; отсутствующий .runtime означает использование настроек сборки по умолчанию. Это не относится к .crc : runtime его не открывает.

Как проверить лимит внутреннего пула

Packed-профили MSBuild по умолчанию имеют фиксированный внутренний пул 384 MiB. Секция [runtime] не изменяет этот лимит: она только запрашивает размеры кэшей и число workers внутри уже скомпилированного пула. Для предварительной проверки используйте формулу:

требуется ~= размер `ampr_assets.index`
+ размер резидентного `AMPRIDX3`
+ `decoded_cache_bytes`
+ `physical_cache_bytes`
+ 32 pipeline-окна * 2 MiB
+ `workers` * 1 MiB
+ 32 MiB защитного резерва

При четырёх `workers` pipeline-окна, `worker scratch` и защитный резерв вместе занимают 100 MiB ещё до манифеста, `AMPRIDX3` и кэшей. Для пула 384 MiB остаётся:

на оба кэша <= 384 MiB - 100 MiB - манифест - `AMPRIDX3` - запас

Оставляйте дополнительно хотя бы 8–16 MiB запаса на выравнивание, небольшие служебные выделения и возможный рост итогового манифеста. Проверяйте фактический размер `ampr_assets.index` после сборки, а не только прогноз до неё. Например, для профиля с 7 439 926 chunks переход с 16-байтовой записи `AMPRPAK3` на 12-байтовую `AMPRPAK4` уменьшает манифест примерно с 113,54 до 85,16 MiB. При `AMPRIDX3` 0,03 MiB, `decoded`-кэше 96 MiB, `physical`-кэше 64 MiB и четырёх `workers` требуется около 345,19 MiB, то есть остаётся примерно 38,81 MiB. Даже `decoded`-кэш 120 MiB требует около 369,19 MiB и оставляет 14,81 MiB. Файл `.crc` в этот расчёт не входит, потому что `runtime` его не загружает.

Во время запуска `runtime` сначала удерживает память под pipeline-окна, `worker scratch` и резерв, затем выделяет `physical`- и `decoded`-кэши. Если запрошенный кэш не помещается, его размер последовательно уменьшается вдвое вплоть до допустимого минимума; поэтому профиль у самой границы может фактически получить заметно меньший кэш. Ищите реальные размеры в строках журнала `apr.pack.physical-cache.init` и `apr.pack.cache.init`.

Если 384 MiB объективно недостаточно, размер меняется при сборке, например через `MSBuild`-свойство `AmprInternalAmmPoolSize=0x20000000u11` для 512 MiB. Это увеличивает `.bss` и потребление CPU-памяти эмулятором; сначала предпочтительнее укрупнить неоправданно мелкие блоки или уменьшить кэши, а повышение лимита обязательно проверять на консоли вместе с памятью самой игры.

9. Проверить на консоли

Проверьте как минимум запуск, новое прохождение, загрузку разных сохранений, переходы между уровнями, `fast travel`, кат-сцены, `streaming`, смену языка, `DLC`,

повторные посещения, отмену загрузки и штатное завершение игры.

Как проводить A/B-тест

Сравнивайте loose и packed на одной консоли, одном носителе, одной версии игры, одном сохранении и в одинаковой последовательности действий. Первый холодный проход и повторный тёплый проход оценивайте отдельно. Делайте не менее трёх сопоставимых прогонов каждого варианта: единичный результат легко искажается системным кэшем и фоновой активностью.

Не выбирайте профиль только по коэффициенту сжатия. Сопоставляйте время загрузки и frame-time outliers со следующими показателями последнего полного snapshot:

Наблюдение	Что проверить перед изменением профиля
Высокий physical/delivered	Слишком крупные блоки для random-доступа, layout и эффективность physical-кэша
Много decoded-cache eviction при доказанном повторном доступе	Достаточен ли фактически выделенный decoded-кэш и не заняли ли память слишком мелкие chunks
Низкий cache hit без повторных чтений	Увеличение кэша не поможет; streaming и second-touch admission ожидаемо обходят его
Высокие queue peaks и p95/p99	Класс нагрузки, число реально занятых workers, задержку backing AIO и конкуренцию lanes
Рост LZ4-нагрузки без уменьшения physical bytes	Сравнить store /RAW и пороги слабого сжатия

Одновременно меняйте один существенный параметр: геометрию блоков, layout, lanes, кэш или workers. Иначе нельзя определить причину результата.

Для диагностических замеров используйте готовую отладочную packed-версию эмулятора без подробной бинарной трассы и verbose-журналов.

После каждого одинакового сценария сохраняйте ampr_emu.log отдельно. Не суммируйте периодические snapshots: они накопительные. p50/p95/p99 — верхние границы корзин, а worker time — прошедшее время работы, не CPU time.

```
python tools/analyze_ampr_log.py "profiling/title/runs/baseline/ampr_emu.log" --tail 0
```

Финальный комплект сравните в предоставленной релизной packed-версии с исходным loose-вариантом по времени загрузки, frame-time, памяти и корректности.

Отладочная версия сама имеет накладные расходы и не подходит для итоговой оценки.

Типичные ошибки и первая проверка

Симптом или строка журнала	Вероятная причина	Что сделать
Не совпадают fileId и путь	Трасса или профиль построены с <code>ampr_emu.index</code> от другой версии <code>/app0</code>	Вернуть полный исходный каталог, пересоздать индекс и трассы как один набор
<code>apr.pack.index.missing</code>	<code>ampr_assets.index</code> не перенесён по ожидаемому пути	Проверить комплект и путь внутри <code>/app0</code>
<code>apr.pack.index.invalid</code>	Повреждён манифест, превышен runtime-лимит или формат не соответствует сборке	Повторить <code>verify / inspect</code> , проверить количества и пересобрать тем же инструментарием
<code>apr.pack.profile.invalid</code>	<code>.runtime</code> повреждён или относится к другому build ID	Пересоздать <code>.runtime</code> из того же TOML и манифеста либо удалить его для <code>compiled defaults</code>
Том отсутствует либо не проходит build ID/CRC	Смешаны сборки или копирование оборвалось	Повторно перенести индекс и все тома одним комплектом, сверить контрольные суммы

Симптом или строка журнала	Вероятная причина	Что сделать
<code>apr.pack.caches.disabled reason=no-runtime-reserve</code>	Манифест и runtime-бюджет не помещаются во внутренний пул	Укрупнить мелкие блоки, уменьшить кэши/workers или обоснованно увеличить build-time pool
Размеры в <code>apr.pack.physical-cache.init</code> или <code>apr.pack.cache.init</code> меньше TOML	Runtime уменьшил кэш из-за нехватки непрерывной памяти	Использовать фактически выделенные размеры при анализе и увеличить запас
Packed-файл существует, но чтение в игре падает	Возможен mmap , прямой или иной неперехваченный путь	Вернуть оригинал и добавить последнее точное правило loose
LZ4/decode error в runtime	Повреждён том, смешаны сборки либо повреждены descriptor/данные	Не продолжать тест; выполнить offline verify с соответствующим .crc и заменить весь комплект
<code>invalid or missing chunk CRC sidecar</code> в Python	<code>ampr_assets.index.crc</code> отсутствует, повреждён или относится к другому build ID	Вернуть соответствующий .crc из той же сборки либо пересобрать/сконвертировать комплект

10. Те же действия без GUI

Ниже пример для двух трасс. Пути выполняются из корня репозитория:

```
$caseRoot = Join-Path (Get-Location) 'profiling/title'

python tools/ampr_pack_profile.py generate `
  --trace "$caseRoot/traces/startup/ampr_commands.bin" "$caseRoot/traces/startup/ampr_emu.index" `
  --trace "$caseRoot/traces/travel/ampr_commands.bin" "$caseRoot/traces/travel/ampr_emu.index" `
  --output "$caseRoot/profiles/title.toml" `
  --report "$caseRoot/profiles/title.report.md" `
  --metrics "$caseRoot/profiles/title.metrics.json" `
  --pattern-mode exact --cache-sim --cache-max-touches 250000 --overwrite
```

После команды вручную дополните TOML правилами для остальных файлов и папок.

Например:

```
[[rule]]
action = "compress"
include = ["dlc/**", "localization/ru/**", "levels/not_visited/**"]
block_size = "64KiB"
layout = "mixed"
force_pack = true
```

Оставьте safety-exclusions последними, потому что последнее совпавшее правило побеждает. Затем соберите и обязательно выполните проверку с `--root` :

```
python tools/ampr_pack.py pack --root "$caseRoot/app0" --ampr-index "$caseRoot/app0/ampr_emu.index"
python tools/ampr_pack.py verify --index "$caseRoot/packed/ampr_assets.index" --root "$caseRoot/app0"
python tools/ampr_pack.py inspect --index "$caseRoot/packed/ampr_assets.index"
python tools/ampr_pack.py list --index "$caseRoot/packed/ampr_assets.index" --json
```

Команда `pack` печатает прогресс в `stderr` , а итоговый JSON — отдельно в `stdout` . Это сохраняет совместимость со скриптами, которые разбирают JSON. Поле `crc` указывает созданный `offline-sidecar`.

Поле `loose_paths` итогового JSON перечисляет все файлы, которые не вошли в паки и должны остаться физически доступными в `/app0` . Сюда входят пути, оставленные `loose` по умолчанию, явными правилами и автоматической проверкой сжимаемости. Чтобы отключить прогресс, добавьте `--no-progress` .

list показывает, что осталось loose. Опция `--self-contained` запрещает автоматически возвращать явно выбранные несжимаемые файлы в loose и сохраняет их RAW-блоками, но она **не выбирает автоматически все файлы игры**.

Старый AMPRPAK3 можно преобразовать без перепакровки .pak -томов:

```
python tools/convert_ampr_pack_v3.py --index "$caseRoot/packed-v3/ampr_assets.index" --output "$caseRoot/packed-v4/ampr_assets.index"
```

Конвертер потоково создаёт 12-байтовую таблицу chunks AMPRPAK4 и отдельный `ampr_assets-v4.index.crc` , проверяя CRC старого манифеста. Build ID сохраняется, а AMPRDAT3-тома остаются без изменений. Сначала проверьте новый индекс командой `verify` ; затем заменяйте рабочий индекс только при остановленной игре.

Сначала просмотрите план удаления — без `--confirm` команда ничего не меняет:

```
python tools/ampr_pack.py remove-packed-sources --index "$caseRoot/packed/ampr_assets.index" --root "$caseRoot/packed"
```

После резервного копирования и проверки списка выполните удаление. Параметр `--confirm` разрешает необратимое изменение, а `--remove-empty-dirs` дополнительно удаляет только опустевшие каталоги:

```
python tools/ampr_pack.py remove-packed-sources --index "$caseRoot/packed/ampr_assets.index" --root "$caseRoot/packed"
```

Изменение размеров блоков, layout или состава файлов требует перепакровки и нового `verify` . Изменение только `[runtime]` выполняется без перекомпрессии:

```
python tools/ampr_pack.py runtime-config --index "$caseRoot/packed/ampr_assets.index" --config "$caseRoot/packed/ampr_runtime.toml"
python tools/ampr_pack.py inspect --index "$caseRoot/packed/ampr_assets.index"
```

Сохраните итоговый TOML, `*.include.txt` , `.runtime` , `.crc` , отчёт и результаты тестов как профиль конкретной версии игры. После обновления игры или изменения ресурсной раскладки соберите новые трассы, снова проверьте ручное покрытие и пересоздайте паки.